Proyecto Final — Planificación Académica

Sistema de Gestión Académica y Rutas de Aprendizaje · Estructuras de Datos 2026-2

Contenido: 1. Cómo ejecutar · 2. Manual de usuario · 3. Justificación de estructuras de datos · 4. Diagrama de clases UML · 5. Diagrama de flujo

1. Cómo ejecutar

Requisito: Java 17 o superior. Siempre se ejecuta desde la **raíz del repositorio**, porque los paquetes siguen la ruta de carpetas (`Semana8.ProyectoFinal...`).

- **VS Code / IntelliJ:** abrir el repositorio y darle *Run* a `PlanificacionAcademica.java`.
- **Consola (Windows PowerShell):**

```
javac -d out (Get-ChildItem -Recurse Semana8 -Filter *.java).FullName
java -cp out Semana8.ProyectoFinal.PlanificacionAcademica
```

- **Consola (Linux/macOS):** `javac -d out $(find Semana8 -name "*.java")` y luego el mismo `java`.
- **Casos de prueba** (los 7 ejemplos del enunciado y casos extra, 31 verificaciones): `java -cp out Semana8.ProyectoFinal.util.Pruebas`

Organización del proyecto:

```
Semana8/ProyectoFinal/
├── PlanificacionAcademica.java   main con el menu de 25 opciones
├── inscripciones.csv             100 solicitudes para el procesamiento por lotes
├── modelo/         Persona, Estudiante, Profesor, MateriaCursada, Materia,
│                   Horario, Aula, Campus, Facultad, Reporte
├── estructuras/    Historial, NavegadorReportes, Operacion y sus 4 hijas
├── excepciones/    las 7 excepciones del enunciado
├── util/           Consola, DatosDemo, LoteCSV, Pruebas
└── documentacion/  README.md, este documento, diagrama_clases.png y diagrama_flujo.png
```

2. Manual de usuario

Al iniciar, el programa pregunta si se cargan los **datos de prueba**: 40 estudiantes (2024001 a 2024040; Ana María Gómez, 2024001, tiene las notas del Ejemplo 6), 3

profesores y 8 materias con pre-requisitos y horarios. CALC101 tiene 3 cupos y PROG201 tiene 2, para probar la cola de espera. Las 5 facultades, las 5 aulas (101, 102, 201, LAB-1, AUD-1) y los 5 edificios existen siempre.

Opción	Qué hace
1 Registrar estudiante	ID, nombre, email (valida el formato) y semestre.
2 Buscar por ID	Búsqueda en el HashMap: datos, promedio y materias actuales.
3 Listar estudiantes	Tabla con todos los registrados.
4 Eliminar estudiante	Pide confirmación; los cupos que libera pasan al primero de cada cola. *Deshacible.*
5 Crear materia	Código, nombre, cupos, créditos, facultad y profesor opcional.
6 Agregar pre-requisito	No permite repetidos ni ciclos.
7 Mostrar pre-requisitos	Recorre la lista enlazada.
8 Inscribir estudiante	Valida pre-requisitos y cruces de horario; si no hay cupo, va a la cola de espera. *Deshacible.*
9 Cancelar inscripción	El cupo pasa automáticamente al primero de la cola. *Deshacible.*
10 Mostrar cola de espera	Posiciones en orden de llegada.
11 Reservar horario	Aula, día (0 = Domingo … 6 = Sábado), hora, duración y materia opcional. Si la materia ya tenía horario, se lo cambia. *Deshacible.*
12 Liberar horario	Las materias de ese bloque quedan sin horario. *Deshacible.*
13 Consultar disponibilidad	Estado de la hora y agenda del día.
14 Agregar conexión	Opcionalmente crea un edificio; muestra la matriz de adyacencia.
15 Ruta más corta	Dijkstra: camino, metros por tramo y total.
16 Registrar nota	Acepta coma o punto; si ya existía, la corrige. *Deshacible.*
17 Ver reporte	1) Completo; 2) reprobadas; 3) promedio de un semestre.
18 Navegador de reportes	`A` atrás, `D` adelante, `S` salir.
19 / 20 Deshacer / Rehacer	Revierte o vuelve a aplicar la última operación.
21 Procesar CSV	Con Enter usa `inscripciones.csv`. Todo el lote se deshace con un solo paso.

Opción	Qué hace
23 / 24 / 25 Consultas	Listar materias; listar aulas (por nombre o capacidad); estudiantes y profesores.
22 Salir	

Formato del CSV: una línea por solicitud, `idEstudiante,codigoMateria` (encabezado opcional). Las líneas mal escritas se reportan como fallidas.

Guion sugerido para el video (10 min):

1. Opciones 1, 2 (con `2024001` y con `2024999`) y 3.
2. Opción 8 con `2024002` a `2024006` en `CALC101`, luego opción 10, y opción 9 cancelando `2024002`: el cupo pasa a Laura Díaz.
3. Opción 19 (Juan vuelve y Laura regresa a la cola), opción 20, y otra vez opción 20 para ver el error.
4. Opción 8 con `2024008` en `PROG201` (le falta un pre-requisito).
5. Opción 11 en el aula `101`, día 1, hora 8, duración 2; repetir con hora 9 para mostrar el conflicto.
6. Opción 15 de 0 a 3 (330 m) y opción 14 para agregar un edificio.
7. Opción 17 con `2024001` y opción 18 para navegar.
8. Opción 21 (78 exitosas y 22 fallidas) y opción 19 para revertir el lote.

3. Justificación de estructuras de datos

3.1 Decisión general

- **Arreglos y matrices nativos:** los exige el enunciado.
- **`LinkedList`, `Stack` y `Queue` de Java:** son las clases que se trabajaron en los talleres del curso (semanas 5 y 6). Están probadas, lo que evita errores de punteros, y ofrecen la complejidad esperada: `push`/`pop` y `add`/`poll` en O(1), agregar al final de la lista en O(1). El código usa siempre las operaciones propias de cada estructura: la cola solo se atiende por el frente y la pila solo por el tope.
- **`HashMap` y `TreeMap`:** los exige el enunciado.

3.2 Dónde se usa cada estructura

Estructura	Dónde	Por qué
`boolean[7][24]`	`Aula.ocupado`	Fila = día, columna = hora; consultar o marcar una hora es O(1). Una matriz paralela `String[7][24]` guarda quién tiene la reserva.

Estructura	Dónde	Por qué
`int[N][N]`	`Campus.distancias`	Matriz de adyacencia de un grafo no dirigido (se llenan `[i][j]` y `[j][i]`); 0 = sin conexión.
`Double[10][20]`	`Estudiante.notas`	10 semestres × 20 materias. Se usa `Double` para distinguir "sin nota" (`null`) de 0.0. Una matriz `String[10][20]` guarda el código de cada materia.
`Facultad[5]`	`Facultad.crearFacultades()`	Lista fija, nunca cambia.
`Estudiante[100]`	`PlanificacionAcademica`	Registro en la primera posición libre; al eliminar queda `null` y deshacer lo devuelve a la misma posición.
`HashMap<String, Estudiante>`	`indiceEstudiantes`	Búsqueda por ID en O(1) promedio, sin recorrer el arreglo.
`TreeMap<String, Aula>` / `TreeMap<String, Materia>`	`aulas` / `materias`	Siempre ordenadas por nombre o código. El orden por capacidad se hace con ordenamiento por inserción.
`LinkedList`	Pre-requisitos, historial e inscritos	Tamaño variable; se agrega al final y se recorre para validar.
`Queue`	`Materia.colaEspera` y lote CSV	FIFO: el cupo es para quien llegó primero; las solicitudes se atienden en el orden del archivo.
`Stack` ×2	`Historial`	Deshacer y rehacer (LIFO).
`Stack` ×2	`NavegadorReportes`	Atrás y adelante, como en un navegador web.
`Stack`	Dijkstra, ciclos de pre-requisitos, lote	Invertir el camino, recorrido en profundidad y deshacer el lote al revés.

3.3 Deshacer / Rehacer

`Operacion` es una clase **abstracta** con `deshacer()` y `rehacer()`. Cada hija guarda una "foto" del estado **antes** y **después** del cambio:

- `OperacionInscripcion`: inscribir, cola de espera, cancelar y eliminar estudiante.
- `OperacionHorario`: matrices de las aulas y horario de las materias.
- `OperacionNota`: la casilla de la matriz y la nota anterior.

- `OperacionLote`: todas las operaciones del CSV.

Deshacer pone la foto de antes y rehacer la de después. Así, si al cancelar el cupo pasó al primero de la cola, al deshacer ese estudiante vuelve a su misma posición. Toda operación nueva vacía la pila de rehacer.

3.4 Excepciones y validaciones

Las 7 excepciones del enunciado heredan de `Exception` y se manejan de forma útil:

- `PreRequisitoNoAprobadoException`: se muestra qué pre-requisito falta.
- `HorarioConflictivoException`: se muestra la agenda del día.
- `CupoLlenoException`: el estudiante pasa a la cola de espera.
- `ColaDeEsperaVaciaException`: se informa si hay cupos.
- `ArchivoInvalidoException`: se permite probar otro archivo.
- `EstudianteNoEncontradoException` y `PilaDeshacerVaciaException`: se muestra el mensaje.

Los datos que no existen o están repetidos (una materia o un aula inexistente, un ID duplicado) usan `IllegalArgumentException`. También se validan: los ciclos de pre-requisitos, los cruces de horario entre materias del estudiante y que una reserva con conflicto no marque ninguna hora.

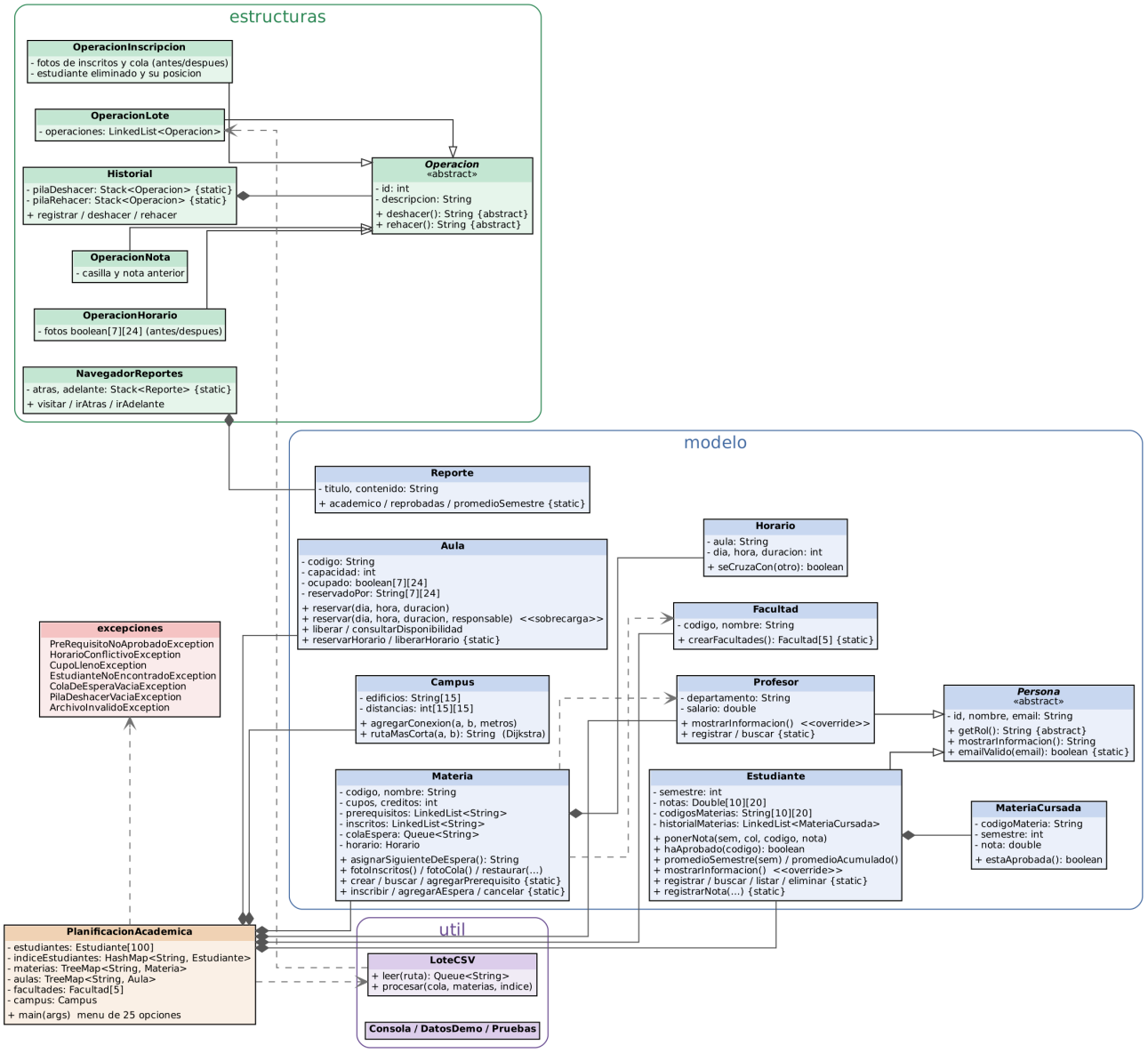
3.5 Programación orientada a objetos

- **Herencia:** `Persona` (abstracta) → `Estudiante`, `Profesor`; y `Operacion` (abstracta) → 4 hijas.
- **Encapsulamiento:** atributos privados con getters; las listas y colas se entregan como copias.
- **Polimorfismo:** `mostrarInformacion()` y `deshacer()`/`rehacer()` están sobrescritos en cada hija; la sobrecarga está en `Aula.reservar(dia, hora, duracion)` y `Aula.reservar(dia, hora, duracion, responsable)`.

Nota sobre el enunciado: en el Ejemplo 6 hay 6 notas y 2 son menores que 3.0, así que son **4 aprobadas**, no 5. Los promedios del ejemplo (4.10, 3.00 y 3.55) sí coinciden.

4. Diagrama de clases UML

Semana8.ProyectoFinal - Diagrama de clases



5. Diagrama de flujo

Muestra el flujo general del programa (menú y manejo de errores), el proceso de inscripción con sus validaciones y la cola de espera, y cómo funcionan cancelar y deshacer.

Semana8.ProyectoFinal - Diagrama de flujo

